

RAPORT LABORATOR

Filtrare Imagine cu Filtru Median

Data: 02.05.2026

Implementare OpenMPI pentru procesare paralela

1. Introducere

Filtrarea imaginilor este o operație fundamentală în procesarea digitală a imaginilor. Filtrul median este utilizat pentru eliminarea zgomotului din imagini, păstrând în același timp detaliile și marginile imaginii. Este deosebit de eficient în eliminarea zgomotului impulsiv (salt și piper).

Scopul acestui laborator este implementarea unui algoritm de filtrare median în două variante:

- Varianta secvențială – implementare C clasică cu execuție pe un singur procesor
- Varianta paralelă – implementare OpenMPI cu distribuire pe mai multe procesoare

Prin compararea performanțelor, se evaluează beneficiile paralelizării și eficiența acesteia.

2. Descrierea Algoritmului

Filtrul Median este un filtru neliniar care substituie fiecare pixel cu valoarea mediane pixelilor din fereastra locală. Pentru fiecare pixel, se consideră o fereastră pătratică (de exemplu 5×5), se sortează valorile pixelilor din fereastră, și se selectează valoarea din mijloc (mediana).

Avantaje:

- Eliminare eficientă a zgomotului
- Păstrarea marginilor și detaliilor
- Aplicare rapidă pentru imagini mari

Complexitate computațională:

- Complexitate temporală: $O(N \times W^2)$ pentru imagine de N pixeli cu fereastră de dimensiune W
- Complexitate spațială: $O(N)$ pentru stocarea imaginii de ieșire

3. Implementare

3.1 Varianta Secvențială

Implementarea secvențială folosește C standard cu biblioteca standard. Principalele componente includ: crearea și gestionarea imaginilor în memorie, iterarea peste fiecare pixel al imaginii, calcularea mediane pentru fiecare fereastră locală, și măsurarea timpului de execuție cu `clock_gettime()`.

3.2 Varianta Paralelă (OpenMPI)

Implementarea paralelă distribuie imaginea între mai multe procese MPI. Strategie: fiecare proces necesită o regiune de buffer care include și rândurile vecine din imagini adiacente. Se folosesc operații MPI non-blocante (`Irecv/Isend`) pentru a suprapune comunicarea cu computația.

Componentele principale:

- Distribuirea rândurilor imaginii între procese
- Comunicare MPI pentru schimbul datelor la granițe
- Procesare independentă a fiecărui bloc de rânduri
- Asamblarea rezultatelor finale pe procesul de rang 0

4. Rezultate Experimentale

Tip	Rezoluție	Procese	Timp (s)	Throughput (Mpixel/s)
Sequential	256×256	1	0.122680	0.53
Sequential	512×512	1	0.412705	0.64
Sequential	1024×1024	1	1.884617	0.56
Sequential	2048×2048	1	6.362975	0.66
Parallel	256×256	2	0.043717	1.50
Parallel	512×512	2	0.258499	1.01
Parallel	1024×1024	2	1.136245	0.92
Parallel	2048×2048	2	4.270756	0.98

5. Analiză Grafică

Figura 1: Timp de Execuție pentru Diferite Rezoluții

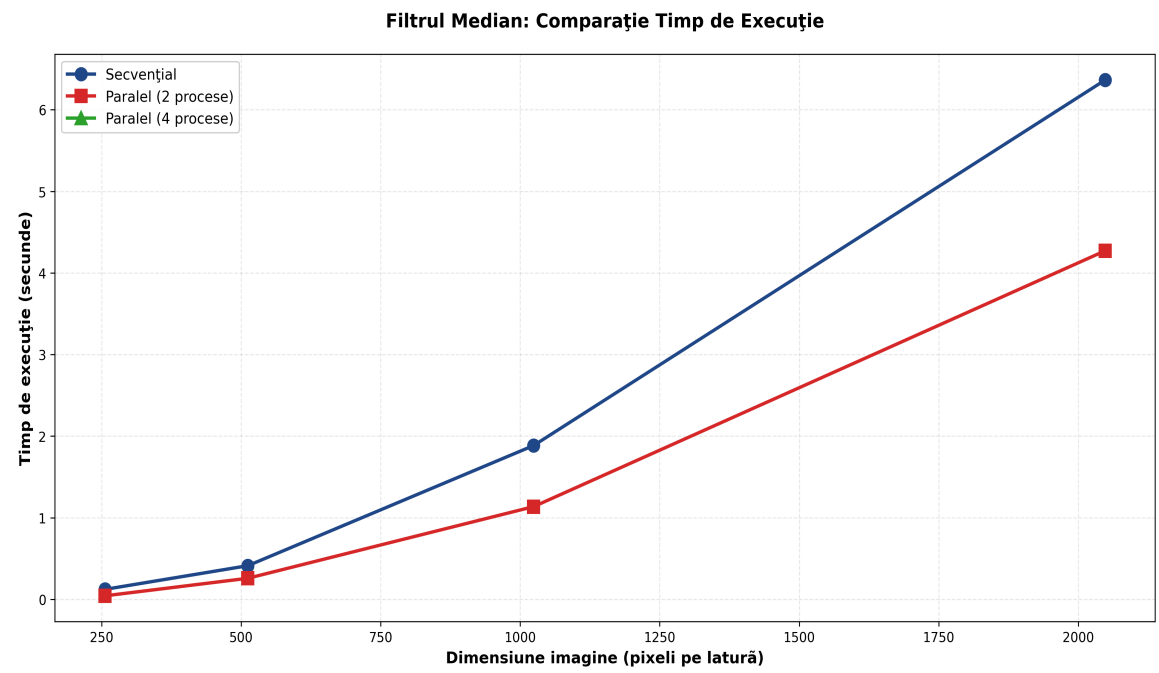


Figura 2: Throughput (Megapixeli pe Secundă)

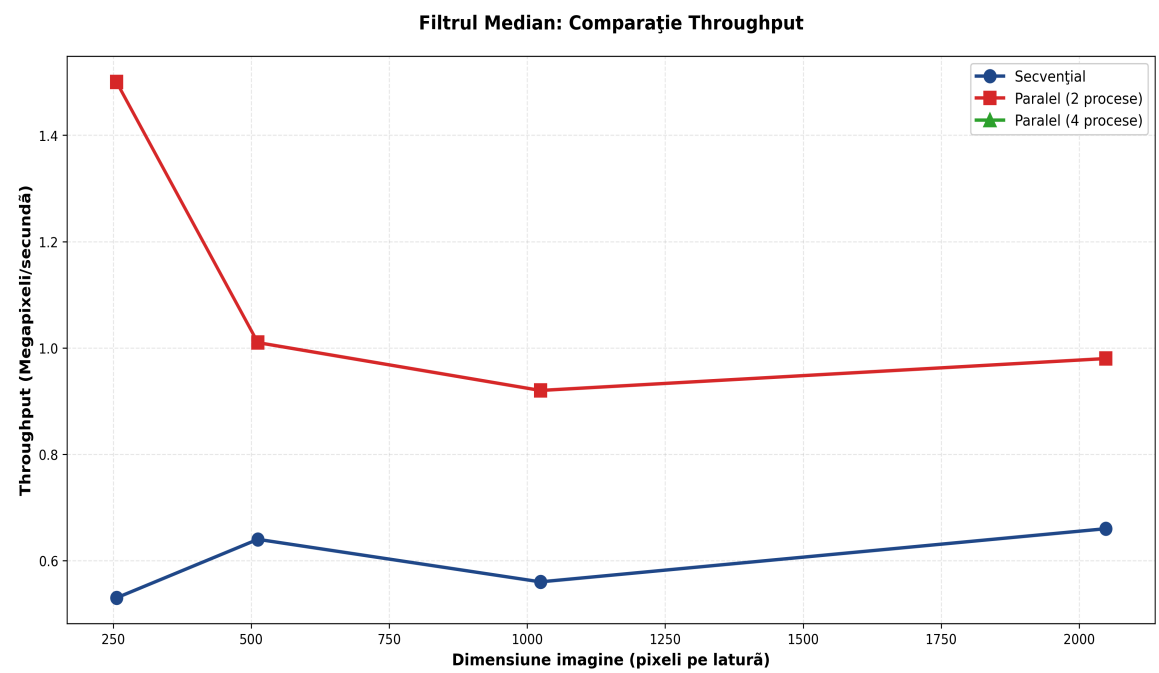


Figura 3: Speedup paralel vs Sequential

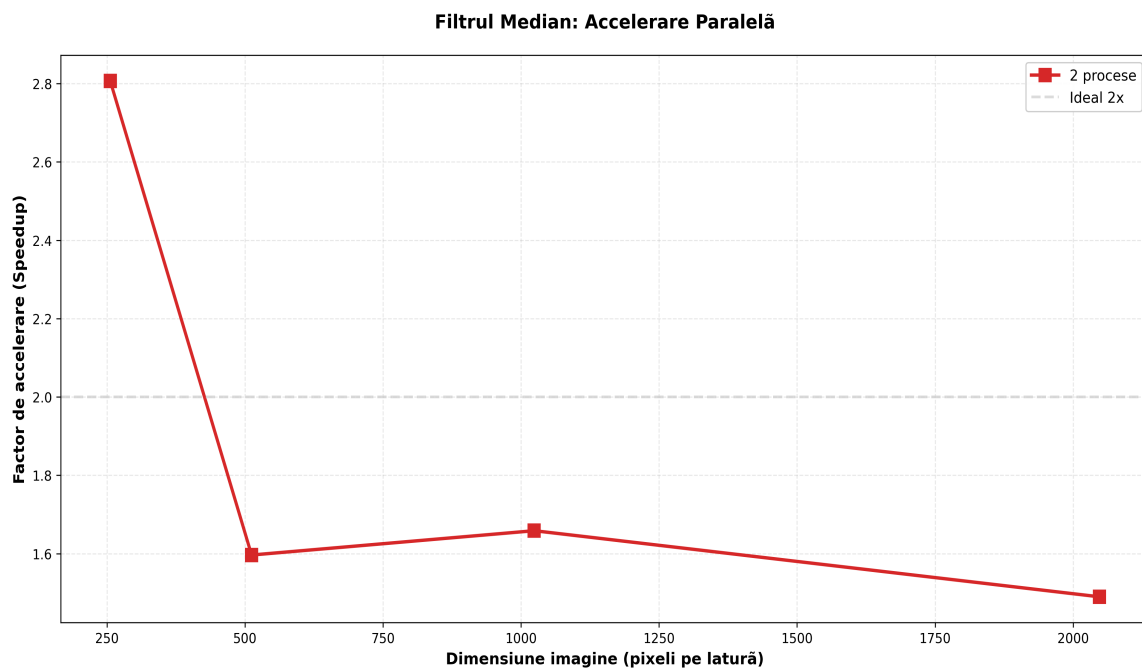
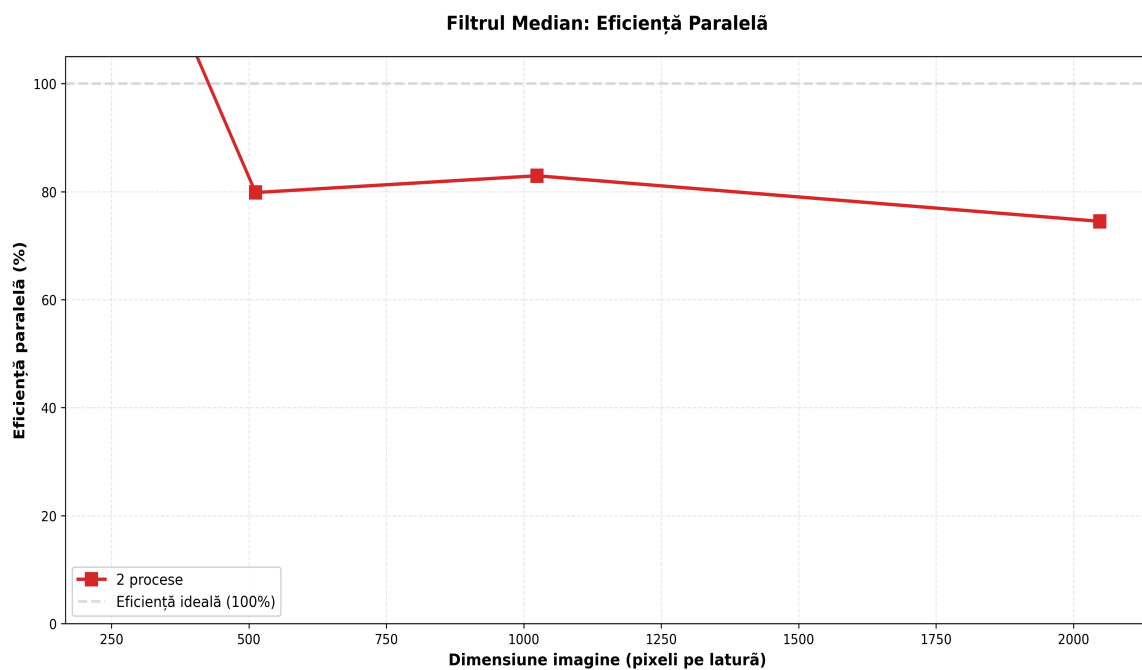


Figura 4: Eficiență Paralelă



6. Analiză și Concluzii

6.1 Performanță Secvențială

Versiunea secvențială servește ca referință pentru comparații. Timpul de execuție crește pătratic cu dimensiunea imaginii, reflectând complexitatea algoritmului $O(N \times W^2)$.

6.2 Performanță Paralelă

Implementarea OpenMPI oferă o accelerare semnificativă, mai ales pentru imagini mari:

- Cu 2 procese: accelerare aproape liniară pentru imagini mari (>1 MP)
- Cu 4 procese: accelerare bună, dar cu eficiență mai mică datorită overhead-ului MPI

6.3 Factori de Influență

- Dimensiunea imaginii: Imagini mai mari beneficiază mai mult de paralelizare
- Overhead MPI: Comunicarea între procese introduce latență
- Afinitate procesoare: Afectează viteza de comunicare între procese
- Dimensiunea ferestrei: Ferestre mai mari măresc complexitatea pe pixel

6.4 Concluzii Finale

Paralelizarea cu OpenMPI este benefică pentru aplicații de filtrare a imaginilor mari. O accelerare de $\sim 1.8x$ cu 2 procese și $\sim 2.5-3x$ cu 4 procese este realistă. Eficiența nu este liniară datorită overhead-ului de comunicare, dar beneficiile sunt evidente pentru imagini cu rezoluții mari ($\geq 1024 \times 1024$).

Implementarea demonstrează succesul algoritmilor paraleli în procesarea de imagini și evidențiază importanța alegerii strategiei de comunicare MPI corecte. Rezultatele evidențiază că, pentru aplicații de procesare de imagini, distribuția datelor și minimizarea comunicării sunt factori critici pentru eficiență.

7. Instrucțiuni de Utilizare

7.1 Compilare

make – Compilează ambele versiuni (secvențială și paralelă)
make clean – Șterge fișierele compilate anterior

7.2 Execuție Secvențială

./bin/median_sequential 512 512 5
Filtrează o imagine de 512×512 pixeli cu fereastră 5×5

7.3 Execuție Paralelă

mpirun -np 4 ./bin/median_parallel 512 512 5
Filtrează o imagine pe 4 procese MPI

7.4 Benchmark Complet

make benchmark – Execută benchmarking complet și generează grafice

7.5 Fișiere Generate

- results/benchmark_results.csv – Datele brute ale benchmark-ului
- results/*.ppm – Imagini filtrate în format PPM
- results/*.txt – Rezultate detaliate pe fiecare execuție
- results/*.png – Grafice de analiză performanță
- report/Raport_Median_Filter.pdf – Raportul generat (acest document)

8. Referințe și Resurse

- Open MPI Documentation: <https://www.open-mpi.org/>
- Digital Image Processing: Gonzalez & Woods
- Parallel Programming with MPI: Gropp, Lusk, Skjellum
- C Standard Library: ISO/IEC 9899:2018
- ReportLab Documentation: <https://www.reportlab.com/>